MSARCH – Project Proposal

Alzheimer's Disease Drug Combination Search: Benchmarking SIMD, CUDA, and Simulated Quantum Computing Approaches

Roan Cedric Campo Adrian Denzel Macayan Luis Andrew Madridijo

October 2026

Abstract

Alzheimer's disease (AD) involves many biological processes at once, so treatment is expected to need combinations of drugs, often ones already approved for other conditions. Computationally, choosing k drugs from n candidates is a search over $\binom{n}{k}$ combinations. We follow a network-medicine rule for good combinations: each drug's protein targets should lie close to the AD disease proteins, while the drugs' target sets should stay separated from one another. We encode this rule as a single integer-valued score ("energy") over every combination, in a quadratic form suited to both annealing and QAOA, and search for its minimum in three ways: (1) An exhaustive GPU search checks every combination and computes the true minimum, which serves as ground truth. (2) A simulated-annealing search is implemented as sequential C++, as an AVX2SIMD kernel written in assembly, and as a CUDA kernel. All three implementations use the same per-chain random streams and fixed-point integer arithmetic, ideally returning identical solutions. Any timing difference therefore reflects the underlying hardware rather than the algorithm. We report throughput and speedup over the sequential baseline as n, k, and the number of parallel chains grow. (3) Moving away from traditional computing, we further implement a Quantum Approximate Optimization Algorithm (QAOA) running on a simulated quantum computer implemented via NVIDIA cuQuantum. In benchmarking, we measure how close it gets to the exact minimum, along with its simulation time and memory, which grow exponentially with n. As a limitation, we show that the results are for computational hypotheses for later experimental testing, not treatment recommendations, and because the quantum computer is simulated on classical hardware, we cannot conclude quantum speedup.

1 Introduction

1.1 The Case for Combination Search in Alzheimer's Disease

Alzheimer's disease (AD) is the most common cause of dementia. Its two hallmarks are plaques of amyloid-β outside nerve cells and tangles of tau protein inside them, and in lesser cases: inflammation, synaptic loss, vascular damage and altered brain metabolism. Drugs aimed at a single target are shown to be ineffective against AD, and the approved anti-amyloid antibodies only modestly slow decline, so previous work suggests that several processes may need to be treated at once. The annual review of the AD drug-development pipeline reports that 35% of the drugs now in AD clinical trials are *repurposed*, meaning they are already approved for another indication, and that these drugs make up almost half of Phase 2 agents [1]. Repurposed drugs give us a pool of approved candidates with known safety profiles, which makes a search over their combinations practical. Clinical trials of

combinations are slow and expensive, because the number of trial arms grows with every drug added, so a computational method that prioritizes a short list of promising combinations can notably aid in generating hypotheses prior to these programs.

1.2 A Network Rule for Drug Combinations

We based our proposal under the framework of network medicine treating the cell's proteins as a graph, the human *interactome*. Proteins linked to the same disease cluster together in this graph, forming a *disease module* [2, 3]. A drug acts on a few proteins, its *targets*. Drugs whose targets are unusually close to the disease module are more likely to treat that disease [4].

For combinations, Cheng, Kovács and Barabási [5] looked at how the targets of two drugs relate to each other and to the disease module. Of the six possible patterns, only one went with combinations that actually work in patients: both drugs are close to the disease module, but their targets are in different parts of the network. They dub this observation as *complementary exposure*.

In plain words

Picture the interactome as a city map, with AD living in one neighborhood. Each drug is a delivery truck that stops at a few buildings. A good *single* drug stops near the AD neighborhood. A good *group* of drugs stops near the AD neighborhood, but each truck covers a different street, so they do not waste effort on the same buildings. Choosing the best team of, say, three trucks out of thirty is the problem this project attempts to solve.

1.3 Motivation as to how this is a Computational Problem

Choosing k drugs out of n means searching $\binom{n}{k}$ combinations. This is trivial for $n = 20$, $k = 3$ (1,140 combinations), but it grows quickly: $\binom{100}{5} \approx 7.5 \times 10^7$, $\binom{1000}{5} \approx 8 \times 10^{12}$, and $\binom{1000}{10} \approx 2.6 \times 10^{23}$. We observe that the search consists of many independent, regular operations, and each of our three methods stresses a different part of the machine:

- **Exact search** scores every combination, and each score is independent of the others, so one GPU thread can check a slice of them. We expect it to be limited by decoding combination indices and reading the score table.
- **Simulated annealing** runs many independent search chains. Every accepted step inside a chain adds one row of the score table to a length-n array and subtracts another, which is what a `ymm` register (eight 32-bit integers) and `vpaddd`/`vpsubd` are for. Only accepted steps do this work, so we expect the speedup to depend on the acceptance rate, which tends to fall as the temperature drops.
- **Quantum simulation** stores 2^n complex amplitudes and, for every gate, applies the same small matrix to every group of two or four of them. Each gate streams the whole state through memory, so we expect it to be limited by memory bandwidth, and memory capacity caps n at about 30 qubits on our single GPU.

SIMD instructions (AVX2) and GPUs (CUDA) suit this kind of regular, data-parallel work, but not every part speeds up equally. Part of our contribution is measuring where the speedup comes from and where it stops, and comparing all three methods in the range of n where all of them can run.

1.4 The Quantum-based Addition

Extending beyond traditional computing. our scoring function has single terms and pair terms only, which is the shape of a Quadratic Unconstrained Binary Optimization (QUBO) problem [6, 7]. QUBOs are the standard input for quantum optimization algorithms such as QAOA [8]. We have no quantum computer, but NVIDIA cuQuantum [9] simulates one with an NVIDIA GPU that we are able to access. Because our instances are small enough to solve exactly, we can measure precisely how close simulated QAOA gets to the true best answer. A recent study suggests network-based drug-combination prediction as a QUBO and solved it with simulated quantum annealing for four other diseases [10]; we found no study that applies QAOA to AD drug combinations.

1.5 Expected Deliverables

1. A documented and checked scoring function (QUBO) for AD drug combinations, built from publicly-accessible drug data.
2. An exact GPU search over all valid combinations, which gives the reference answer.
3. One simulated-annealing algorithm written in three approaches (sequential C++, AVX2 assembly, CUDA), with identical results, and an execution-time comparison.
4. QAOA, simulated with CUDA-Q and cuStateVec, measured against the exact answer.

Several larger ideas were considered and left out to keep the project appropriate within the demands of one trimester; they are listed with reasons and motivations in Section H.

1.6 Relationship to Sister Work: Network-Screening project

This project replaces the optional Module 5 of the sister project and can run independently. It needs two things for each candidate drug: its closeness score z and its list of targets. It can take these from the sister project's output, or compute them for a hand-picked candidate list with a small script of its own (Decision D1, **??**). The two projects agree on one table format (Section F), so either source can be plugged in without changing the underlying solvers.

2 Review of Related Literature

2.1 Network Proximity and the Disease Module

Network medicine studies disease through the network of molecular interactions [2]. Menche et al. [3] showed that disease proteins form connected modules in the incomplete human interactome and defined the *network separation* s_{AB} of two protein sets, which is negative when the sets overlap in the network and positive when they are apart. Guney et al. [4] turned closest-distance proximity into a drug screen, standardizing it against degree-matched random target sets to obtain a z-score; drugs with significantly negative z are enriched for known treatments. Cheng et al. [11] applied the approach to cardiovascular disease with population-based validation.

2.2 Network-based Drug Combinations

Cheng, Kovács and Barabási [5] combined the two measures: for a drug pair (A, B) and disease module S, they considered the proximity of each drug to S and the separation s_{AB} between the drugs' targets. Using approved combinations for hypertension and cancer, they identified that only

the *complementary exposure* pattern (both drugs close to S, targets separated, $s_{AB} \geq 0$) correlated with therapeutic effect, and that the drug–drug proximity z-score by itself could not distinguish approved combinations from random pairs. This finding is our objectives' scientific basis: reward proximity of each drug to the AD module and penalize overlap between the selected drugs' targets.

2.3 Network Proximity in Alzheimer's Disease

Fang et al. [12] modeled AD endophenotype modules, screened drugs by proximity, and used insurance-claims data to associate sildenafil use with lower AD incidence. Later pipelines ranked drugs for different pathological stages of AD [13] and combined proximity ranking with heart-rhythm (Long QT) risk screening [14]. The AD genes used to build modules come from large genome-wide association studies [15], and the annual pipeline reviews [16, 1] list drugs currently in AD trials, including repurposed agents. These studies rank single drugs; we found none that optimizes AD combinations with the complementary-exposure principle.

2.4 QUBO Formulations and Constraints

Many selection problems can be written as QUBOs, $\min_{\mathbf{x}\in\{0,1\}^n} \mathbf{x}^\top Q\mathbf{x}$ [6, 7]. Constraints such as "exactly k items" are usually added as quadratic penalties, $A(\sum_i x_i - k)^2$. This is convenient but has known costs: the penalty couples every pair of variables, making Q dense, and a penalty weight that is too small allows invalid solutions while one that is too large flattens the energy landscape that heuristics and quantum algorithms rely on [6].

2.5 Parallel simulated annealing

Simulated annealing (SA) [17] is the standard classical heuristic for QUBO and Ising problems. A single SA chain is inherently sequential, since every step depends on the previous state, so parallel implementations usually run many independent chains (replicas) or exchange states between chains at different temperatures (parallel tempering) [18]. Isakov et al. [19] showed how careful data layout, precomputed acceptance tables and multi-spin coding make SA on Ising models much faster on CPUs. Counter-based random-number generators such as Philox [20] produce the same random stream on CPU and GPU from a (key, counter) pair, which makes cross-platform reproducibility possible.

2.6 QAOA and Constraint-preserving Mixers

QAOA [8] alternates a cost layer $e^{-i\gamma H_C}$ with a mixing layer $e^{-i\beta H_M}$, starting from the uniform superposition, and tunes the $2p$ angles classically. Hadfield et al. [21] generalized it to the Quantum Alternating Operator Ansatz, in which the mixer and initial state are chosen so the state never leaves the set of valid solutions. For "exactly k ones" constraints, *XY mixers* preserve the number of ones (the Hamming weight) [22], and a natural initial state is the Dicke state, the equal superposition of all weight-k strings, which can be prepared deterministically with $O(kn)$ gates [23]. Cook, Eidenbenz and Bärtschi [24] studied this ansatz on Maximum k-Vertex Cover, a problem with the same cardinality structure as ours, comparing ring and complete-graph XY mixers. Zhou et al. [25] described parameter-initialization heuristics that make QAOA training at larger depth p practical.

2.7 Quantum Approaches to Drug Combinations

Ramos, Coutinho and Magano [10] built a QUBO from the complementary-exposure principle and sampled it with simulated quantum annealing, testing it on diabetes mellitus, rheumatoid arthritis, asthma and brain neoplasms against experimentally validated combinations. Their interactome and drug–target data come from Cheng et al. [5], and their code is public. They did not study AD, gate-model QAOA, constraint-preserving mixers, or parallel classical solvers.

2.8 GPU Simulation of Quantum Circuits

NVIDIA cuQuantum [9] features cuStateVec (state-vector simulation) and cuTensorNet (tensor-network simulation). It is a simulation library, not a quantum processor or an optimizer. CUDA-Q [26] is NVIDIA's quantum programming framework; its default `nvidia` target runs single-GPU state-vector simulation through cuStateVec, with an `fp64` option for double precision [26]. CuStateVec's documented requirements include compute capability 7.0 or higher (Volta, Turing, Ampere, Ada, Hopper) [27] which we have access to.

2.9 Synthesis and Research Gap

Work	What it does	AD	Comb.	Parallel classical	Gate QAOA
Guney et al. [4]	Proximity z-score drug screen	–	–	–	–
Cheng et al. [5]	Complementary exposure for combinations	–	✓	–	–
AD proximity studies [12, 13, 14]	AD single-drug repurposing	✓	–	–	–
Isakov et al. [19]	Optimized CPU SA for Ising	–	–	✓	–
Cook et al. [24]	XY-mixer QAOA, cardinality constraint	–	–	–	✓
Ramos et al. [10]	Combination QUBO, simulated quantum annealing	–	✓	–	–
Sister project	GPU proximity screen for AD	✓	–	✓	–
This work	AD combination QUBO; SA on scalar, AVX2 and CUDA; exact GPU reference; QAOA vs. exact	✓	✓	✓	✓

Table 1: How this project relates to prior work

Prior work work and exists distinct from AD solutions or computational modeling: a validated network principle for combinations, AD-specific proximity studies, fast parallel annealing, constraint-preserving QAOA, and one quantum-annealing study of combination QUBOs outside AD. What remains to be missing is (a) a carefully validated AD combination QUBO, (b) a controlled comparison of one combination-search algorithm across scalar, SIMD and SIMT hardware, and (c) a measurement of gate-model QAOA against exact optima on these biology-derived instances.

3 Problem Statement

3.1 Problem

Network proximity can shortlist AD drug candidates, and complementary exposure computes what a good pair looks like. In turning this into a recommended *combination* of k drugs, we define a search problem whose size grows as $\binom{n}{k}$. Three questions are therefore open: how to turn closeness, overlap and combination size into one score without inventing biological numbers; how much one search algorithm gains from SIMD and from a GPU, and why; and how well a quantum algorithm (QAOA), simulated on a GPU, finds the best combination compared with the exact answer.

3.2 Proposed Solution

We build the system as five modules. Each module has fixed inputs and outputs, so we can build and test them separately (Table 2). Modules 1–4 are the project core; Module 5 is the research extension and its novelty.

Module	Input	Output	Runs on
1. Candidate preparation	Interactome, drug–target list, AD genes, candidate list; *or* the sister project's table	Candidate table (targets, z_i) and a shortlist of n drugs	CPU (Python)
2. Problem construction	Shortlist, network distances	Rewards w_i, penalties P_{ij}, QUBO and Ising files	CPU (Python)
3. Exact search	w, P, k	The best combination, how many combinations tie for best	CPU and GPU
4. Simulated annealing	w, P, k, settings	Best combination per run, run statistics, timings	CPU scalar, AVX2, GPU
5. QAOA (extension)	Ising file	Sampled combinations and their scores	GPU (CUDA-Q + cuStateVec)

Table 2: Pipeline modules. Every solver reads the same problem file and writes the same result format (Section F).

3.3 Research questions and hypotheses

RQ1 (main): How much do SIMD and the GPU help?

How do sequential C++, AVX2 and CUDA versions of the *same* simulated-annealing algorithm compare in speed, as the number of candidates n grows? We measure the time for one search chain, the number of steps per second when many chains run at once, and the time until the exact best answer is found.

Hypothesis: The GPU wins once enough independent chains run to keep it busy (hundreds or more). AVX2 wins for a single chain. Inside a chain, the speedup is limited because only *accepted* steps do the vectorized array update, and late in the search most steps are rejected.

RQ2 (main): Is the scoring function right?

Does our QUBO always pick a valid combination of exactly k drugs, equal to the best combination found by exact search, and how does the overlap dial λ change which drugs are chosen?

Hypothesis: With the penalty weight set above the bound we prove in Section B, the QUBO

and exact answers always agree. As λ grows, the chosen combinations move from "the k drugs closest to AD" toward complementary combinations.

RQ3 (extension): How good is QAOA?
On instances small enough to solve exactly, how often does simulated QAOA return the best combination, how many of its answers are valid k-drug combinations, and how does this change with circuit depth p and problem size n?
Hypothesis: Most of QAOA's answers will be invalid, because only a tiny share of all yes/no patterns have exactly k ones ($\binom{n}{k}/2^n$; 0.1% at $n = 20$, $k = 3$). Quality will improve with p but remain well below exact search.

RQ4 (supporting): Where does the time go?
How is total run time split between data preparation, problem construction, solving and checking? Answered from timing logs at no extra cost.

RQ1 and RQ2 are the course core and do not depend on the quantum part. RQ3 is the main research question for a possible conference paper.

3.4 Objectives

General objective. To build a checked scoring function for AD drug combinations and compare exact, SIMD, GPU and simulated-quantum ways of finding the best combination.

Specific objectives.

1. Build a reproducible candidate table and shortlist from public data, with clear documentation where every input comes from.
2. Build the QUBO and Ising files and prove, then test, that they return the right answer.
3. Implement exact search on CPU and GPU and confirm they agree.
4. Implement one simulated-annealing algorithm in sequential C++, AVX2 assembly and CUDA; show that all three make identical decisions; compare their execution times.
5. Run QAOA in CUDA-Q on the simulator and measure its answers against the exact search.

3.5 Scope and limitations

- **Public data only.** No patient data.
- **Hypotheses, not recommendations.** A high-scoring combination is a computer-generated idea. It is not evidence that the combination is effective, synergistic or safe.
- **Access to a upper mid-range GPU.** A GeForce RTX 5070Ti with 16 GB of GDDR7 VRAM. This limits the quantum simulation to about 30 qubits (one qubit per drug). It is sufficient for the classical solvers at our sizes.
- **We cannot claim any quantum speedup.** The quantum computer is simulated, and at these sizes exact search is fast. The quantum part measures answer quality only.
- **Weak biological validation.** Very few AD drug combinations have trial results, so we check whether chosen combinations follow the complementary-exposure pattern, not whether they work.

- **Cut for time.** Safety and side-effect data, splitting large problems into QAOA-sized pieces, a constraint-preserving QAOA variant, and several extra performance experiments were left out. Section H lists each one, why it was cut, and how it may be added in the future.

3.6 Significance

The proposed study observes one algorithm on three kinds of hardware and explains where data-level parallelism helps and where a step-by-step dependency may limit it. Furthermore, it gives a realistic, biology-based test problem for quantum optimization, with exact answers to compare against, and the first QAOA study of AD drug combinations that we are aware of.

4 Methodology

4.1 Description of the Proposed Project

4.1.1 Inputs

All inputs are publicly accessible. We record the versions and checksum so our results can be reproduced.

- **Human interactome.** The protein–protein interaction graph compiled by Cheng et al. [5], restricted to its largest connected piece. Using the same graph as [10] makes our results comparable with theirs. If both projects run, the sister project's BioGRID graph [28] can be used instead.

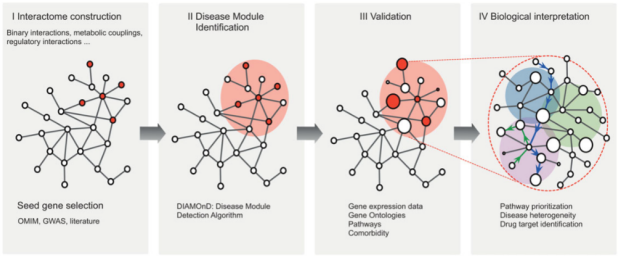


Figure 1: Example of identifying a network module in the Human interactome.

- **Drug–target list.** The drug–target table of Cheng et al. [5], filled in from ChEMBL [29] or DGIdb [30] for any candidate it lacks. Only targets present in the interactome are kept.
- **AD disease module.** The same definition as the sister project: genes from the largest recent AD genetic study [15] plus well-known AD genes (e.g., *APP*, *PSEN1*, *PSEN2*, *APOE*, *MAPT*).
- **Candidate list** (Decision D1). Either the sister project's ranked output, or a hand-picked ("curated") list: repurposed drugs from the 2025 and 2026 AD pipeline reviews [16, 1], the four approved symptomatic AD drugs (donepezil, rivastigmine, galantamine, memantine), and drugs suggested by published AD network studies [12, 13, 14]. Antibody drugs whose targets are not in the interactome are left out.
- **Scaling pool.** For speed experiments only: every drug in the drug–target table with at least one target in the interactome (expected: roughly a thousand).

- **Settings.** Combination size $k \in \{2, 3, 4\}$ for biological results (up to 8 for speed tests); overlap weight λ; number of SA chains and steps; random seeds.

We note that the solvers does not consider genuine biology principles or theories. Instead, they see a list of n numbers (how close each drug is to AD), an $n \times n$ table (how much each pair of drugs overlaps), and the number k. Everything biological is simply a label attached to those numbers.

More precisely, the pipeline answers one question: *which k drugs together cover the AD module best without overlapping each other?*

1. **Score each drug.** Take drug i's closeness score z_i (Section B). Only drugs with $z_i < 0$ (closer than chance) are kept. The reward is $w_i = -z_i$, capped and scaled to the range 0–1.
2. **Score each pair.** Compute the separation s_{ij} between the targets of drugs i and j [3]. Overlapping pairs ($s_{ij} < 0$) get a penalty $P_{ij} = -s_{ij}$, scaled to 0–1; all other pairs get 0.
3. **Find the best team.** Find the set $\mathcal{S}$ of exactly k drugs with the lowest energy

$$E(\mathcal{S}) = -\sum_{i \in \mathcal{S}} w_i + \lambda \sum_{\{i,j\} \subseteq \mathcal{S}} P_{ij}. \tag{1}$$

   The first sum rewards closeness to AD; the second punishes overlap. Low energy therefore means complementary exposure [5].

For three drugs A, B, C the energy is: minus (reward of A + reward of B + reward of C), plus λ times (overlap of A–B + overlap of A–C + overlap of B–C). Pick the three drugs that make this number as small as possible.

(i) Processes parallelized with SIMD (AVX2) and SIMT (CUDA).

1. **Exact search over all combinations**. The $\binom{n}{k}$ combinations are numbered in order and split into equal slices, one per GPU thread. Each thread finds its first combination from its slice number, walks through its slice, and remembers its best. The threads then combine their bests with a minimum reduction. A sequential C++ version gives the cross-check and the timing baseline.
2. **Simulated annealing**. Each chain holds exactly k drugs and repeatedly tries swapping one chosen drug for one unchosen drug. A helper array g of length n makes the score change of any swap an $O(1)$ calculation (Section C). When a swap is accepted, g is updated with $g \leftarrow g + \lambda(P_{:,j} - P_{:,i})$, an array add and subtract over n elements.
   - **AVX2**: this update is a hand-written assembly loop using `ymm` registers, eight 32-bit integers per instruction.
   - **CUDA**: thousands of chains run at once. Version 1 uses one thread per chain.
3. **QAOA simulation** (Module 5). The cuStateVec library applies each quantum gate to all 2^n numbers of the simulated state in parallel on the GPU. We do not write this kernel, but we time it and measure its memory use.

(ii) Related Works and How We Differ. We acknowledge that the complementary exposure [5] and its quantum-annealing QUBO [10] inspires the scoring idea. Optimized simulated annealing [19] and QAOA [8] suggests our solver framework. Our implementation differs in that it:

- targets AD, and proves and tests that the QUBO gives the right answer instead of tuning its settings by trial and error;

- searches only valid combinations in the classical solvers, so no penalty setting is needed there;
- writes *one* SA algorithm for scalar, SIMD and GPU hardware with shared random numbers and integer arithmetic, so the three versions make identical decisions and their run times can be compared fairly;
- measures QAOA against the exact best answer on every instance, instead of reporting quantum results alone.

Step	How the work is split	Limited by
Exact search (GPU)	Combinations numbered $0, \ldots, \binom{n}{k} - 1$ and split into equal slices, one per thread. Each thread decodes its first combination, steps to the next one in order, and re-scores only the drugs that changed. Warp and block minimum reduction of (energy, combination number, tie count).	Arithmetic
SA, AVX2	Per chain: drawing the random swap and computing its score change is scalar code. Accepted swaps update g with `vmovdqa`/`vpaddd`/`vpsubd` over eight lanes, unrolled four times.	Share of steps accepted; memory bandwidth for large n
SA, CUDA	One thread per chain; g in the thread's local memory.	Uncoalesced memory access
QAOA	cuStateVec over 2^n numbers; the angle-tuning loop runs on the CPU.	GPU memory

Table 3: Parallelization plan. Pseudocode and memory layouts are in Section C.

(iii) Parallelization plan. Data moves to the GPU once and results come back only once.

(iv) Processes not parallelized.

- Data Preprocessing and ETL: Downloading data, matching gene and drug identifiers, building the interactome and target lists.
- Computing closeness scores for the curated list when the sister project's output is not used (a small Python script; not part of the contribution).
- Shortest-path distances among target proteins and the separation table s_{ij} (breadth-first search from a few thousand proteins at most; fast enough on the CPU).
- Building the QUBO and Ising files and the annealing temperature schedule.
- Building QAOA circuits and running the angle-tuning optimizer (COBYLA), which calls the GPU simulator.
- Result analysis and plotting.

4.1.2 Outputs

1. **Problem files**: candidate table, separation and penalty tables, QUBO and Ising files, each with a record of where its numbers came from.
2. **Best AD combinations** for each k and λ, with the exact best answer, how many combinations tie for best, and the runners-up.

3. **Execution-time comparison**: sequential C++ vs. AVX2 vs. CUDA for SA, and CPU vs. GPU for exact search, across n and k, with speedup charts.
4. **Correctness report**: CPU and GPU exact search agree; all SA versions make identical decisions; QUBO and Ising scores agree for every yes/no pattern on small problems.
5. **QAOA results**: how often QAOA finds the best combination, the share of valid answers, and simulator time and memory, for each depth p and size n.

5 Proposed Target Implementation Platform

Component	Specification
Parallel paradigms	SIMD: AVX2 (256-bit `ymm` registers) in hand-written x86-64 assembly (NASM) for the inner loop, called from C++; the team already has x86 and `xmm`/`ymm` experience. A C++ intrinsics version of the same loop is kept as a cross-check. SIMT: CUDA C++.
GPU	NVIDIA GeForce RTX 5070 Ti: Blackwell (GB203), compute capability 12.0, 70 SMs with 8,960 CUDA cores, 16 GB GDDR7 on a 256-bit bus, 896 GB/s, 48 MB L2 cache [**?**]. GeForce cards are slow at 64-bit floating point, so all GPU code uses 32-bit integers (deterministic mode) or 32-bit floats.
CPU (baselines)	Intel Core Ultra 7 265KF (Arrow Lake): 8 performance cores and 12 efficiency cores, 20 threads (no hyper-threading), up to 5.5 GHz boost, 30 MB L3 and 36 MB L2 in total [**?**]; [RAM: size, DDR5 speed]. It supports AVX2 and FMA; AVX-512 is not available and not assumed. Same machine as the GPU. Timing runs are pinned to a performance core, and the clock speed is recorded, so that the efficiency cores do not distort the baselines.
OS	Linux (Ubuntu 22.04/24.04). Windows with WSL2 is acceptable for the C++/CUDA core; CUDA-Q is distributed for Linux, so QAOA runs either natively on Linux or under WSL2. GPU visibility under WSL2 is checked with `nvidia-smi` and a CUDA-Q test run before any experiments.
Toolchain	CUDA Toolkit 12.8 or newer (the first release that can compile for compute capability 12.0, `sm_120`) with an NVIDIA driver from the 570 series or newer, GCC/Clang with C++17, NASM, CMake; the Philox random-number generator [20] written once in a header that both CPU and GPU code include.
Quantum stack	CUDA-Q (Python, CUDA 12 build) with the `nvidia` target, which simulates on a single GPU through cuQuantum's cuStateVec [26, 9]. cuStateVec supports the Blackwell GPU family from version 1.8 (cuQuantum 25.03) [27], so we use a recent CUDA-Q release and test it on this card first. If installation fails: CUDA-Q's CPU simulator for small n, or cuQuantum's Python interface directly.
Analysis	Python 3.11+ (NumPy, SciPy, pandas, NetworkX for small reference checks, matplotlib).
Profiling	Nsight Systems and Nsight Compute (recent versions with Blackwell support) on the GPU; `perf` and Intel VTune on the CPU, with hardware counters read on native Linux where available.
Repository	GitHub: source, build and benchmark scripts, frozen data manifests (not the licensed raw data), the parallel-algorithm discussion, and the sequential-vs-parallel timing results required for Milestone 3.

Table 4: Target platform.

The RTX 5070 Ti is a consumer gaming card. That is enough for every classical part of this project, because the problems are small in memory (a 2,000-drug penalty table is 16 MB, which fits

in the GPU's 48 MB L2 cache and in the CPU's 30 MB L3 cache). It is the limiting factor for the quantum simulation: simulating n qubits needs 2^n complex numbers, so each extra qubit doubles the memory. In single precision a 30-qubit state takes 8 GB, so 16 GB runs out at roughly 30 qubits (about 29 in double precision).

References

[1] J. L. Cummings *et al.*, "Alzheimer's disease drug development pipeline: 2026," *Alzheimer's & Dementia: Translational Research & Clinical Interventions*, 2026. Published May 5, 2026. TODO: volume, article number, full author list.

[2] A.-L. Barabási, N. Gulbahce, and J. Loscalzo, "Network medicine: a network-based approach to human disease," *Nature Reviews Genetics*, vol. 12, no. 1, pp. 56–68, 2011.

[3] J. Menche, A. Sharma, M. Kitsak, S. D. Ghiassian, M. Vidal, J. Loscalzo, and A.-L. Barabási, "Uncovering disease-disease relationships through the incomplete interactome," *Science*, vol. 347, no. 6224, p. 1257601, 2015.

[4] E. Guney, J. Menche, M. Vidal, and A.-L. Barabási, "Network-based in silico drug efficacy screening," *Nature Communications*, vol. 7, p. 10331, 2016.

[5] F. Cheng, I. A. Kovács, and A.-L. Barabási, "Network-based prediction of drug combinations," *Nature Communications*, vol. 10, p. 1197, 2019.

[6] F. Glover, G. Kochenberger, and Y. Du, "Quantum bridge analytics I: a tutorial on formulating and using QUBO models," *4OR*, vol. 17, pp. 335–371, 2019.

[7] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.

[8] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm." arXiv:1411.4028, 2014.

[9] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa, *et al.*, "cuQuantum SDK: A high-performance library for accelerating quantum science," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2023. arXiv:2308.01999.

[10] D. Ramos, B. Coutinho, and D. Magano, "Network-based prediction of drug combinations with quantum annealing." arXiv:2512.20199, 2025. Code: `https://github.com/dmrapk/Drug-Combinations-using-Quantum-Annealers`. The sister specification cites a 2026 journal version in Quantum Machine Intelligence; TODO: confirm and update.

[11] F. Cheng, R. J. Desai, D. E. Handy, R. Wang, S. Schneeweiss, A.-L. Barabási, and J. Loscalzo, "Network-based approach to prediction and population-based validation of in silico drug repurposing," *Nature Communications*, vol. 9, p. 2691, 2018.

[12] J. Fang, P. Zhang, Y. Zhou, C.-W. Chiang, J. Tan, Y. Hou, *et al.*, "Endophenotype-based in silico network medicine discovery combined with insurance record data mining identifies sildenafil as a candidate drug for Alzheimer's disease," *Nature Aging*, vol. 1, pp. 1175–1188, 2021.

[13] TODO: authors, "Network proximity-based computational pipeline identifies drug candidates for different pathological stages of Alzheimer's disease," *Computational and Structural Biotechnology Journal*, 2023.

[14] TODO: authors, "Prioritizing repurposable drugs for Alzheimer's disease using network-based analysis with concurrent assessment of Long QT syndrome risk." `https://pmc.ncbi.nlm.nih.gov/articles/PMC12337645/`, 2025. PMID: 40791766.

[15] C. Bellenguez, F. Küçükali, I. E. Jansen, L. Kleineidam, S. Moreno-Grau, N. Amin, *et al.*, "New insights into the genetic etiology of Alzheimer's disease and related dementias," *Nature Genetics*, vol. 54, pp. 412–436, 2022.

[16] J. L. Cummings, Y. Zhou, G. Lee, *et al.*, "Alzheimer's disease drug development pipeline: 2025," *Alzheimer's & Dementia: Translational Research & Clinical Interventions*, vol. 11, no. 2, p. e70098, 2025.

[17] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[18] D. J. Earl and M. W. Deem, "Parallel tempering: Theory, applications, and new perspectives," *Physical Chemistry Chemical Physics*, vol. 7, pp. 3910–3916, 2005.

[19] S. V. Isakov, I. N. Zintchenko, T. F. Rønnow, and M. Troyer, "Optimised simulated annealing for Ising spin glasses," *Computer Physics Communications*, vol. 192, pp. 265–271, 2015.

[20] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, "Parallel random numbers: as easy as 1, 2, 3," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, 2011.

[21] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, p. 34, 2019.

[22] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz," *Physical Review A*, vol. 101, p. 012320, 2020.

[23] A. Bärtschi and S. Eidenbenz, "Deterministic preparation of Dicke states," in *Fundamentals of Computation Theory (FCT 2019)*, vol. 11651 of *Lecture Notes in Computer Science*, pp. 126–139, 2019.

[24] J. Cook, S. Eidenbenz, and A. Bärtschi, "The quantum alternating operator ansatz on maximum k-vertex cover," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, pp. 83–92, 2020. arXiv:1910.13483.

[25] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices," *Physical Review X*, vol. 10, p. 021067, 2020.

[26] NVIDIA Corporation, "NVIDIA CUDA-Q documentation: installation, single-GPU state-vector simulators, and Python API." `https://nvidia.github.io/cuda-quantum/latest/`, 2026. Accessed October 2026.

[27] NVIDIA Corporation, "cuQuantum documentation: Getting started and dependencies." `https://docs.nvidia.com/cuda/cuquantum/`, 2026. Accessed October 2026.

[28] R. Oughtred, J. Rust, C. Chang, B.-J. Breitkreutz, C. Stark, A. Willems, *et al.*, "The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions," *Protein Science*, vol. 30, no. 1, pp. 187–200, 2021.

[29] B. Zdrazil, E. Felix, F. Hunter, E. J. Manners, J. Blackshaw, S. Corbett, *et al.*, "The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods," *Nucleic Acids Research*, vol. 52, no. D1, pp. D1180–D1192, 2024.

[30] S. L. Freshour, S. Kiwala, K. C. Cotto, A. C. Coffman, J. F. McMichael, J. J. Song, *et al.*, "Integration of the drug–gene interaction database (DGIdb 4.0) with open crowdsource efforts," *Nucleic Acids Research*, vol. 49, no. D1, pp. D1144–D1151, 2021.

[31] D. Szklarczyk, R. Kirsch, M. Koutrouli, K. Nastou, F. Mehryary, R. Hachilif, *et al.*, "The STRING database in 2023: protein-protein association networks and functional enrichment analyses for any sequenced genome of interest," *Nucleic Acids Research*, vol. 51, no. D1, pp. D638–D646, 2023.

[32] C. Knox, M. Wilson, C. M. Klinger, M. Franklin, E. Oler, A. Wilson, *et al.*, "DrugBank 6.0: the DrugBank knowledgebase for 2024," *Nucleic Acids Research*, vol. 52, no. D1, pp. D1265–D1275, 2024.

[33] TODO: authors, "DDInter 2.0: an enhanced drug interaction resource with expanded data coverage, new interaction types, and improved user interface," *Nucleic Acids Research*, vol. 53, no. D1, pp. D1356–, 2025.

[34] M. Boustani, N. Campbell, S. Munger, I. Maidment, and C. Fox, "Impact of anticholinergics on the aging brain: a review and practical application," *Aging Health*, vol. 4, no. 3, pp. 311–320, 2008.

[35] F. Meng, Y. Xi, J. Huang, and P. W. Ayers, "A curated diverse molecular database of blood-brain barrier permeability with chemical descriptors," *Scientific Data*, vol. 8, p. 289, 2021.

[36] N. P. Tatonetti, P. P. Ye, R. Daneshjou, and R. B. Altman, "Data-driven prediction of drug effects and interactions," *Science Translational Medicine*, vol. 4, no. 125, p. 125ra31, 2012.

[37] M. Kuhn, I. Letunic, L. J. Jensen, and P. Bork, "The SIDER database of drugs and side effects," *Nucleic Acids Research*, vol. 44, no. D1, pp. D1075–D1079, 2016.

[38] M. Booth, S. P. Reinhardt, and A. Roy, "Partitioning optimization problems for hybrid classical/quantum execution," Tech. Rep. 14-1006A-A, D-Wave Systems, 2017.

A Glossary

This project mixes biology, optimization, parallel computing and quantum computing. Nobody on the team needs to know all four. This glossary gives just enough of each term to read the rest of the document. Terms are grouped by who needs them most.

A.1 Biology (everyone)

Interactome
A graph whose nodes are human proteins and whose edges are known physical interactions between them. About 15,000–20,000 nodes.

Target
A protein that a drug binds to. Most drugs have one to a few dozen targets.

Disease module
The group of proteins linked to a disease (here, AD). They tend to sit close together in the interactome.

Proximity score z_i
How close drug i's targets are to the AD module, compared with random look-alike drugs. Negative means closer than chance; more negative is better. Computed by the sister project or by our small reference script.

Separation s_{ij}
How far apart drug i's targets are from drug j's targets. Negative means the two drugs hit overlapping parts of the network.

Complementary exposure
The pattern of a good combination: each drug is close to the disease, but the drugs' targets do not overlap.

A.2 Optimization (everyone)

Combination size k
How many drugs we pick (2, 3 or 4 for biological results).

Energy E
The score of a combination. *Lower is better.* Rewards are negative energy; penalties are positive energy.

Reward w_i
How much we like drug i on its own (from z_i).

Penalty P_{ij}
How much we dislike picking drugs i and j together (from s_{ij}).

λ (lambda)
A dial that sets how much overlap matters compared with closeness to AD.

Exact search / enumeration
Try every possible combination and keep the best. Always correct, but only possible when the number of combinations, $\binom{n}{k}$, is small enough.

Simulated annealing (SA)
A search method that starts from a random combination and repeatedly tries small changes (here: swap one chosen drug for an unchosen one). Changes that improve the energy are always kept; changes that make it worse are sometimes kept, more often early on ("high temperature") than later ("low temperature"), so the search does not get stuck.

Chain / replica
One independent run of SA. Running many chains with different random numbers and keeping the best result is the main source of parallelism.

Sweep
n proposed swaps, one "pass" over the problem size.

A.3 Parallel computing (everyone)

SIMD lane
One slot in a vector register. A 256-bit `ymm` register holds eight 32-bit lanes, so one instruction (e.g. `vpaddd`) does eight additions.

Kernel
A function that runs on the GPU, launched with many threads at once.

Thread, warp, block
A GPU thread is one worker. 32 threads form a *warp* and execute the same instruction together, like a 32-lane SIMD unit. Warps are grouped into *blocks* that share fast on-chip *shared memory*.

Coalesced access
When the 32 threads of a warp read 32 neighboring memory words, the GPU combines them into one fast transaction. Scattered reads are much slower.

Reduction
Combining many values into one (e.g. finding the minimum energy across all threads).

Philox
A random-number generator that turns a (key, counter) pair into random bits. Same inputs give the same outputs on CPU and GPU, which is how all three versions draw identical random numbers.

A.4 Quantum (owner of the quantum part; others may skip)

QUBO
"Quadratic Unconstrained Binary Optimization": minimize a function of yes/no variables that has single terms (x_i) and pair terms ($x_i x_j$) only. Our energy is already in this shape; the "exactly k drugs" rule is added as a penalty term to make it a QUBO.

Ising model
The same problem written with variables $+1/-1$ instead of $0/1$. Quantum software expects this form. Converting between the two is a fixed formula (Section B).

Qubit
The quantum version of a bit. One qubit per candidate drug.

State vector
A classical computer simulates n qubits by storing 2^n complex numbers. Each extra qubit doubles the memory; this is why the GPU's 6 GB limits us to about 28 qubits.

QAOA
"Quantum Approximate Optimization Algorithm": a quantum circuit with a few adjustable angles. A classical optimizer tunes the angles so that measuring the qubits tends to give low-energy combinations.

Depth p
How many layers the QAOA circuit has. More layers can give better answers but cost more to simulate and tune.

Shot
One measurement of the circuit, giving one candidate combination.

cuQuantum / cuStateVec / CUDA-Q
NVIDIA's libraries for simulating quantum circuits on a GPU (cuQuantum, whose state-vector part is cuStateVec) and for writing quantum programs (CUDA-Q). They are simulators, not quantum computers and not optimizers.

B Mathematical Formulation

This appendix is mainly for Member 1. Members 2 and 3 only need Equation (5) and the plain-words box below it.

B.1 Network quantities (preserved from the sister project)

Let $G = (V, E)$ be the interactome (largest connected component) and $d(u, v)$ the shortest-path length. For drug i with target set $T_i \subseteq V$ and AD module S, the closest distance and its standardized score are, exactly as in the sister project and [4],

$$d_c(S, T_i) = \frac{1}{|T_i|} \sum_{t \in T_i} \min_{s \in S} d(t, s), \qquad z_i = \frac{d_c(S, T_i) - \mu_i}{\sigma_i}, \tag{2}$$

where μ_i, σ_i come from degree-matched random target sets. This project does not change these definitions; it only consumes z_i. If the scores come from the sister project, the file hash and its configuration (number of random samples, degree-bin rule, AD module version) are recorded with the candidate table (Section F).

For two target sets $A = T_i$, $B = T_j$, the network separation is [3, 5]

$$s_{AB} = \langle d_{AB} \rangle - \frac{\langle d_{AA} \rangle + \langle d_{BB} \rangle}{2}, \tag{3}$$

with $\langle d_{AA} \rangle = \frac{1}{|A|} \sum_{a \in A} \min_{a' \in A \setminus \{a\}} d(a, a')$ and

$$\langle d_{AB} \rangle = \frac{1}{|A| + |B|} \left(\sum_{a \in A} \min_{b \in B} d(a, b) + \sum_{b \in B} \min_{a \in A} d(a, b) \right). \tag{4}$$

Single-target drugs. $\langle d_{AA} \rangle$ is undefined when $|A| = 1$. We set it to 0 and flag the pair [PROPOSED].

B.2 Rewards

- **Eligibility.** Drug i is eligible if it has at least one target in G, z_i is finite, and $z_i < 0$ [PROPOSED]. Drugs with $z_i \geq 0$ are not closer than chance and are excluded rather than given negative rewards. This fixes a sign problem in the sister project's Appendix A, where $w_i = -z_i$ would be negative for $z_i > 0$.

- **Transform.** $\tilde{w}_i = \min(-z_i,\, z_{\max})$ with $z_{\max} = 10$ by default, to limit the influence of outliers; then $w_i = \tilde{w}_i / \max_j \tilde{w}_j \in (0, 1]$ [PROPOSED]. An alternative, $w_i = -\log_{10} p_i$ from the empirical p-value, is reported as a sensitivity check only.
- **Shortlist.** For biological instances of size n, take the n eligible drugs with the most negative z_i, breaking ties by drug identifier.
- **Missing scores.** A drug without a score is ineligible; it is never imputed.

B.3 Pairwise penalties: what they mean and what they do not

$P_{ij} = \max(0, -s_{ij})$, normalized to $[0, 1]$ by the largest value in the instance. It penalizes *target overlap in the network*, which Cheng et al. found to be associated with non-efficacious pairs [5]. It is not a measure of adverse drug–drug interactions, clinical synergy, or pharmacokinetic interference; those need different evidence and are left out of this project (Section H). No positive synergy reward is used in the core objective, because no reliable AD synergy data exist [PROPOSED].

Density. $P_{ij} > 0$ only for pairs with $s_{ij} < 0$. The share of nonzero entries in the AD instances is measured and reported.

B.4 The constrained problem

With $x_i \in \{0, 1\}$ ($x_i = 1$: drug i selected),

$$\min_{\mathbf{x}\in\{0,1\}^n} f(\mathbf{x}) = -\sum_i w_i x_i + \lambda \sum_{i<j} P_{ij} x_i x_j \quad \text{subject to} \quad \sum_i x_i = k. \tag{5}$$

This is the problem every solver is judged on. The classical solvers solve it directly.

In plain words

Each drug is either in ($x_i = 1$) or out ($x_i = 0$). The energy adds the negative reward of every drug that is in, plus λ times the overlap of every pair of drugs that are both in. We want exactly k drugs in and the lowest possible energy.

B.5 Penalty QUBO

Adding $A(\sum_i x_i - k)^2$ and using $x_i^2 = x_i$ gives $E(\mathbf{x}) = \sum_i a_i x_i + \sum_{i<j} b_{ij} x_i x_j + c$ with

$$a_i = -w_i + A(1 - 2k), \qquad b_{ij} = \lambda P_{ij} + 2A, \qquad c = Ak^2. \tag{6}$$

Every b_{ij} is nonzero, so the QUBO links every pair of drugs even when P is mostly zero.

In plain words

The penalty $A(\text{count} - k)^2$ is zero when exactly k drugs are chosen and grows quickly otherwise. If A is too small, cheating (picking too many or too few drugs) can pay off; if it is too large, QAOA has a harder time telling good valid answers from bad ones. The proposition below gives the smallest safe value.

Penalty bound. Let $w_{\max} = \max_i w_i$ and $R_{\max} = \max_i \sum_{j \neq i} P_{ij}$. Assume $w_i \geq 0$ and $P_{ij} \geq 0$ (guaranteed above).

Proposition. If $A > \max(w_{\max}, \lambda R_{\max})$, every $\mathbf{x}$ with $\sum_i x_i \neq k$ has a single-bit neighbor with strictly lower E. Hence every global minimizer of E satisfies the constraint and minimizes (5).

Proof. Let $m = \sum_i x_i$. If $m > k$, turn off any selected i: f changes by $w_i - \lambda \sum_{j \in \mathcal{S} \setminus \{i\}} P_{ij} \leq w_{\max}$, and the penalty changes by $A[(m-1-k)^2 - (m-k)^2] = A[1 - 2(m-k)] \leq -A$; the total change is below 0. If $m < k$, turn on any unselected i: f changes by $-w_i + \lambda \sum_{j \in \mathcal{S}} P_{ij} \leq \lambda R_{\max}$, and the penalty changes by $A[2(m-k)+1] \leq -A$; again the total is below 0. On the constraint set the penalty is 0, so $E = f$ there. □

This bound is smaller than the one in the sister project's Appendix A ($A > \max_i w_i + \lambda \sum_j P_{ij}$), which matters for QAOA, where an unnecessarily large A compresses the spread of valid energies. Default $A = 1.1 \max(w_{\max}, \lambda R_{\max})$; QAOA experiments also use $2\times$ and $4\times$ the bound. The proposition is checked exhaustively by test T3 (Section G).

B.6 QUBO to Ising

We use $x_i = (1 - z_i)/2$, so computational basis state $|0\rangle$ ($z_i = +1$) means "not selected" and a measured 1 means "selected". Substituting into E:

$$H = \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j + C, \quad h_i = -\frac{a_i}{2} - \frac{1}{4} \sum_{j \neq i} b_{ij}, \quad J_{ij} = \frac{b_{ij}}{4}, \quad C = c + \frac{1}{2} \sum_i a_i + \frac{1}{4} \sum_{i<j} b_{ij}. \tag{7}$$

The offset C is stored with the Ising file so that decoded energies can be compared with f directly.

B.7 Coefficient scaling

The argmin is unchanged by positive scaling. For classical float solvers, coefficients are used as normalized above. For QAOA, all Ising coefficients are divided by $\max(|h_i|, |J_{ij}|)$ so the angle γ has a comparable range across instances; the scale factor is stored. For the deterministic integer mode, w and λP are multiplied by 2^q and rounded, with q chosen so the largest possible $|E|$ of a k-subset stays below 2^{30}; the rounding error per energy is at most $(k + \binom{k}{2}) \cdot 2^{-q-1}$ in normalized units, and test T4 checks that the integer optimum equals the float64 optimum on every small instance.

B.8 Alternative constraint encodings

B.9 Ties, missing data and infeasible outputs

- **Ties.** Exact search returns the optimal energy, the number of tied optima, and the lexicographically smallest optimal combination; all tied optima are listed for biological instances.
- **Invalid outputs.** Only QAOA can return invalid patterns (not exactly k drugs). They are counted as invalid and never silently fixed.
- **Missing pairwise evidence.** Separation is defined for every pair in the connected interactome; if a target set becomes empty after filtering, the drug is ineligible.

C Classical Solver Designs

This appendix is written for Members 2 and 3. It assumes C++ and x86 assembly, but no background in optimization. There are two algorithms (exact search and simulated annealing), and each is

Encoding	Extra variables	Couplings	Validity	QAOA cost per layer
Exact-k penalty (**used** for QAOA)	0	all $\binom{n}{2}$	only at optimum, if A above bound	n R_Z + $\binom{n}{2}$ ZZ; X mixer n R_X
"At most k" with binary slack	$\lceil \log_2(k+1) \rceil$	all pairs incl. slack	only at optimum	more qubits and couplings; not used
Search only valid combinations (our SA and exact search)	0	nnz(P)	always	classical only (**used**)
XY mixer + Dicke state (stretch goal)	0	nnz(P) in cost	always (every gate preserves weight)	n R_Z + nnz(P) ZZ; ring mixer n XY gates or complete mixer $\binom{n}{2}$; Dicke prep $O(kn)$ once

Table 5: Constraint encodings. "nnz(P)" is the number of overlapping pairs.

run in more than one way (sequential, SIMD, GPU). Keep these two ideas separate: "AVX2" and "CUDA" are not algorithms; they are ways of running the same algorithm.

C.1 What every solver reads and writes

- **Reads** the problem file (Section F): n, k, λ, the rewards w (length n), the penalty table P ($n \times n$), and integer copies of both for deterministic mode. P is stored *column by column*, each column padded to a multiple of 8 entries and 32-byte aligned, so a column can be read with aligned `ymm` loads.
- **Keeps** the current combination as two arrays: `in[k]` (chosen drugs) and `out[n-k]` (the rest), plus a position map so a swap takes $O(1)$ time.
- **Writes** one record per run: the best combination (sorted drug indices), its energy as computed by the solver, its energy recomputed by one shared reference function ("the judge"), the seed, counters, and timings. If the two energies disagree, the run is marked invalid.

C.2 Exact search

In plain words

Exact search simply tries every combination of k drugs and keeps the best. For $n = 24$ and $k = 3$ there are only 2,024 combinations; for $n = 1024$ and $k = 3$ there are about 179 million, still fine for a GPU. It is the "answer key" that every other solver is graded against.

Numbering the combinations. List all k-drug combinations in dictionary order, e.g. for $n = 5$, $k = 3$: $\{0,1,2\}, \{0,1,3\}, \{0,1,4\}, \{0,2,3\}, \ldots$, and number them $0, 1, 2, \ldots$. A standard method (the "combinatorial number system") turns a number back into its combination, so each thread can jump straight to the start of its slice.

Walking a slice. Each thread scores its first combination in full, then moves to the next one in dictionary order. Usually only the last drug changes, so the thread keeps the partial energy of the

first $k-1$ drugs and recomputes only the end. It remembers its best energy, the number of that combination, and how many combinations tied with it.

Combining. A warp-level minimum (using `__shfl_down_sync`), then a block-level minimum in shared memory, then a final minimum over blocks on the CPU. The sequential C++ version runs the same code in one loop.

When to use it. Whenever $\binom{n}{k}$ is small enough to finish within the time budget. The actual speed on the GTX 1660 is measured in Week 8.

C.3 Simulated annealing (SA) with swap moves

In plain words

SA starts from a random team of k drugs. At each step it picks one drug on the team and one drug off the team and asks: "would swapping them lower the energy?" If yes, it swaps. If no, it *sometimes* swaps anyway, with a probability that shrinks over time. Early on ("high temperature") it wanders freely; later ("low temperature") it only accepts improvements. The occasional uphill move lets it escape bad local choices. Running many independent searches ("chains") and keeping the best answer is what makes it parallel.

The helper array g. Re-scoring the whole team after every proposed swap would be slow. Instead, every chain keeps, for every drug v (on or off the team),

$$g_v = -w_v + \lambda \sum_{l \in \mathcal{S}} P_{vl}, \tag{8}$$

which is "how much the energy would change if drug v joined the current team." With it, swapping team member i out and outsider j in changes the energy by

$$\Delta E = g_j - g_i - \lambda P_{ij}. \tag{9}$$

This is three memory reads and two subtractions, whatever n is. (The $-\lambda P_{ij}$ corrects for the fact that g_j counted an overlap with i, who is leaving.) After an accepted swap, every g_v changes by $\lambda(P_{vj} - P_{vi})$:

$$g \leftarrow g + \lambda P_{:,j} - \lambda P_{:,i}, \tag{10}$$

which is an add and a subtract over two whole columns of the table. This is the line that SIMD and the GPU speed up.

Temperatures and stopping. Temperatures fall geometrically from T_0 to T_1 over S sweeps: $T_s = T_0(T_1/T_0)^{s/(S-1)}$. T_0 is chosen so that about 80% of uphill steps are accepted at the start, and T_1 so that about 0.1% are at the end; Member 1 computes both once per problem and stores them in the settings file. A run stops after S sweeps, or, in time-to-answer tests, as soon as it reaches the known best energy.

Chains. R independent chains, each with its own chain number, so each draws different random numbers. The run's answer is the best over all chains. Chains never talk to each other.

Algorithm 1 Swap-move SA, one chain (the same logic on every platform)

pick a random starting team of k drugs; compute g; $E \leftarrow$ energy of the team
for sweep $s = 0, \ldots, S-1$ **do** ▷ temperature T_s read from a table
  for $q = 1, \ldots, n$ **do** ▷ one sweep = n proposed swaps
    $(r_0, r_1, r_2, \cdot) \leftarrow$ PHILOX(seed, chain, step) ▷ four random 32-bit numbers
    $i \leftarrow \texttt{in}[(r_0 \cdot k) \gg 32]$; $j \leftarrow \texttt{out}[(r_1 \cdot (n-k)) \gg 32]$
    $\Delta E \leftarrow g_j - g_i - \lambda P_{ij}$
    if $\Delta E \le 0$ **or** $\Delta E < T_s \cdot L[r_2 \gg 20]$ **then** ▷ L: shared table of $-\ln u$ values
      $g \leftarrow g + \lambda(P_{:,j} - P_{:,i})$ ▷ the vectorized step
      swap i and j between `in` and `out`; $E \leftarrow E + \Delta E$
      if $E < E_{\text{best}}$ **then** save E_{best} and `in`
return E_{best}, best team, counters

C.4 Random numbers and deterministic mode

Philox [20] is a function: give it a key (the run seed) and a counter (chain number, step number), and it returns four random 32-bit numbers. Because it has no hidden state, the 1,000th step of chain 7 gets the same numbers on the CPU and the GPU, no matter which thread runs it or in what order. It is written once in a header file used by all versions.

In **deterministic mode**, w, λP, g, E and the temperatures are 32-bit integers, and the acceptance test compares ΔE with T_s times an entry of a 4,096-entry integer table L of $-\ln u$ values (computed in 64-bit to avoid overflow). Every decision is then pure integer arithmetic on identical inputs, so the scalar, AVX2 and CUDA versions must make *exactly* the same accept/reject choices. Test T7 (Section G) compares the first million decisions of 64 chains. If they differ, there is a bug, and the first differing step shows where. A floating-point mode is kept for comparison.

C.5 AVX2 version (Member 3)

The proposal and ΔE stay scalar. The update of g is a loop over n_{pad} integers reading two columns of the table:

```
; void g_update_i32(int32_t* g, const int32_t* col_j, const int32_t* col_i, size_t n_pad)
; rdi = g, rsi = lambda*P[:,j], rdx = lambda*P[:,i], rcx = n_pad (multiple of 32)
.loop: vmovdqa ymm0, [rdi]        ; load 8 x int32 of g
       vpaddd  ymm0, ymm0, [rsi]  ; + lambda*P[:,j]
       vpsubd  ymm0, ymm0, [rdx]  ; - lambda*P[:,i]
       vmovdqa [rdi], ymm0        ; store
       ; ... repeat for ymm1..ymm3 at offsets 32, 64, 96 (unroll x4) ...
       add rdi, 128
       add rsi, 128
       add rdx, 128
       sub rcx, 32
       jnz .loop
       vzeroupper
       ret
```

The floating-point version uses `vsubps` and `vaddps` (or `vfmadd231ps`). The same routine is reused to compute g at the start of a run.

What to expect. Rejected steps do no vector work, so the overall SA speedup is lower than the speedup of the loop itself; for large n the loop is limited by memory bandwidth (three arrays

streamed). Both are measured and explained, not assumed.

Fair comparison. The scalar baseline is compiled with `-O2` and `-fno-tree-vectorize` so the compiler does not secretly vectorize it; a normal `-O3 -march=native` build is reported next to it. All main comparisons are single-threaded.

C.6 CUDA version (Member 2)

Start with v1. It is a plain translation of Algorithm 1 and is enough for the course. Do v2 only after v1 passes test T7.

Kernel	How work is mapped	Synchronization
`exact_search`	One thread per slice of combination numbers; w and P in global memory (for $n \leq 64$, P copied into shared memory first).	Warp and block minimum; final minimum on the CPU.
`sa_v1` (thread per chain)	Thread t runs chain t exactly as in Algorithm 1. Its g, `in`, `out` live in global memory, laid out chain-interleaved ($g[v \cdot R + t]$) so that neighboring threads read neighboring addresses.	None.
`sa_v2` (warp per chain)	32 threads share one chain; 4 chains per 128-thread block. g, `in`, `out` in shared memory (8 KB per chain at $n = 2048$). Lane 0 draws the random numbers and decides; the decision is sent to the other lanes with `__shfl_sync`; on accept, all 32 lanes update g together, lane ℓ handling entries $\ell, \ell + 32, \ldots$, so reads of P's columns are coalesced.	`__syncwarp` only.

Table 6: CUDA kernels.

What to expect. v1 has no idle threads, but each thread's $O(n)$ update touches memory far from its neighbors' (unless the interleaved layout works well). v2 reads memory efficiently, but 31 of 32 lanes sit idle on every *rejected* step, and most late steps are rejected. Which one is faster depends on n; finding the crossover is one of the results (Section E). In both, the speedup comes from running thousands of chains, not from making one chain faster.

Three ways to measure speed. (a) *Latency*: time for one chain to finish, on each platform. (b) *Throughput*: proposed steps per second with R chains, for $R \in \{1, 64, 512, 4096, 16384\}$. (c) *Time to answer*: time until any chain reaches the exact best energy, with each platform using the number of chains that suits it best. The GPU should lose (a) and win (b); (c) says which is faster in practice.

D Quantum Design: QAOA with cuQuantum

This appendix is for Member 1. Others only need the plain-words box and Table 8.

In plain words

QAOA is a recipe for a quantum circuit with a few adjustable angles. Each layer of the circuit first "rewards" low-energy combinations (using our Ising file) and then "mixes" combinations together. A normal computer program tunes the angles: it runs the circuit, measures the average energy, adjusts the angles, and repeats. At the end, we measure the circuit thousands of times; each measurement gives one candidate combination, and we keep the best valid one. On our GPU, the "quantum computer" is a simulation that stores 2^n numbers.

D.1 Who does what

Step	Done by
Build the QUBO and convert it to Ising form	Our Python builder (Module 2)
Write the QAOA circuit	Our CUDA-Q kernel (Python `@cudaq.kernel`)
Simulate the circuit	CUDA-Q `nvidia` target, which calls cuQuantum's cuStateVec on the GTX 1660 [26, 9]
Average energy for a set of angles	CUDA-Q `observe` (exact, since the simulator knows the full state)
Tune the angles	COBYLA optimizer on the CPU (CUDA-Q's optimizers or SciPy)
Draw and check answers	CUDA-Q `sample`; our code decodes and scores each answer with the judge
Real quantum hardware	Not used

Table 7: Roles. cuQuantum is a simulator; it neither builds the problem nor optimizes it.

The CUDA-Q functions named here (`cudaq.kernel`, `cudaq.sample`, `cudaq.observe`, `cudaq.set_target`, `cudaq.set_random_seed`, `cudaq.spin`) are from its documented Python API [26]. Their exact signatures are checked against the installed version in Week 11 and recorded in the repository.

D.2 The circuit

- **Problem**: the scaled Ising form of the penalty QUBO (Section B), with A at 1.1×, 2× and 4× the proven safe value.
- **Start**: every qubit in the $|+\rangle$ state (equal mix of all 2^n patterns) [8].
- **One layer**: an $R_Z(2\gamma h_i)$ rotation on each qubit; a ZZ rotation by $2\gamma J_{ij}$ on every pair (CX, R_Z, CX); then an $R_X(2\beta)$ rotation on each qubit.
- **Depth**: $p \in \{1, 2, 3, 4, 5\}$ layers, each with its own (γ, β).

D.3 Tuning the angles

- $p = 1$: try a 20×20 grid of (γ, β) values, then run COBYLA from the best one.
- $p > 1$: start from the tuned $p - 1$ angles, stretched to p layers (the "interpolation" trick of [25]), plus 2 random starts; COBYLA with at most 300 evaluations.

D.4 Reading the answers

- After tuning, take 8,192 samples ("shots") with a fixed simulator seed.
- Turn each bitstring into a list of chosen drugs, in the documented qubit order. Test T6 checks the order by preparing known patterns.
- An answer is *valid* if it has exactly k ones. Every valid answer is re-scored by the judge.
- QAOA's result is its best valid answer. We do *not* fix invalid answers.

D.5 What we measure

For each instance, exact search gives the best energy E_{opt} and the worst valid energy E_{worst}.

- **Valid share**: fraction of samples with exactly k drugs.
- **Optimum share** P_{opt}: fraction of samples that are a best combination; $1/P_{\text{opt}}$ is the expected number of shots needed to see it once.
- **Quality** of valid samples: $r = (E_{\text{worst}} - \bar{E}_{\text{valid}})/(E_{\text{worst}} - E_{\text{opt}})$, which is 1 if every valid sample is optimal and 0 if all are the worst.
- **Cost**: circuit evaluations, shots, two-qubit gates per layer, simulator time, peak GPU memory.

D.6 Memory limits on the GTX 1660

Simulating n qubits stores 2^n complex numbers: $8 \cdot 2^n$ bytes in single precision, $16 \cdot 2^n$ in double.

n	single precision	double precision	On a 6 GB GTX 1660
16	512 KiB	1 MiB	trivial
20	8 MiB	16 MiB	trivial
24	128 MiB	256 MiB	comfortable
26	512 MiB	1 GiB	comfortable
28	2 GiB	4 GiB	single fits; double to be tested
30	8 GiB	16 GiB	does not fit

Table 8: Memory for the simulated state (arithmetic). The real limit is lower than 6 GB because the CUDA context and the library also need memory; it is measured, not assumed.

Plan: $n \in \{8, 12, 16, 20, 24\}$ in single precision (CUDA-Q's default), with a double-precision check at $n \leq 16$. Larger sizes only if time allows. Each layer has $\binom{n}{2}$ pair gates, and every gate touches the whole state, so run time grows quickly with n.

Why a state-vector simulator. cuQuantum also offers tensor-network simulation, which helps for weakly connected circuits. Our penalty term connects every pair of qubits, so the state vector, which is exact and fits in memory at our sizes, is the better choice.

What the results mean. A bigger simulated circuit is not evidence of quantum advantage. All results are reported as simulator measurements of answer quality.

D.7 Comparing QAOA with classical solvers fairly

The same problem size does not make a fair comparison, because the solvers spend effort differently. We report answer quality against (a) wall time on the same GPU and (b) the number of candidate combinations examined (SA steps vs. QAOA shots $\times$ circuit runs).

D.8 Stretch goal: QAOA that never gives invalid answers

If standard QAOA is finished by Week 12, Member 1 may add a variant that keeps the number of chosen drugs fixed at k throughout: start from the equal mix of all k-drug patterns (a "Dicke state", prepared with the circuit of [23]) and replace the R_X mixer with "XY" gates that swap a 1 and a 0 between neighboring qubits [21, 22, 24]. Every answer is then valid, and the penalty A is not needed. This is listed as future work if not reached (Section H).

E Evaluation Plan

E.1 Two families of test problems

Family	Source	Sizes	Used for
Small (biological)	Curated AD list (D1)	$n \in \{12, 16, 20, 24\}$; $k \in \{2, 3, 4\}$; $\lambda \in \{0, 0.25, 0.5, 1, 2\}$	Checking the scoring function, exact answers, QAOA, biological results
Large (speed)	Scaling pool, scored by the sister project or our script	$n \in \{128, 256, \ldots, 2048\}$; $k \in \{3, 5, 8\}$; $\lambda = 1$	CPU vs. AVX2 vs. GPU speed for SA and exact search

Table 9: Test problem families. Every problem file records its family and where its numbers came from. If neither source can score the scaling pool, randomly generated problems with the same statistics are used and clearly labeled as synthetic.

Reference answers. Small problems: exact search. Large problems: exact search wherever it finishes in time (all $k = 3$ problems; $k = 5$ up to the size the GPU can handle); otherwise the best energy found by very long SA runs, reported as "best known", never as "optimal".

E.2 Small problems

All solvers get the same problem file: exact search (CPU, GPU), SA (scalar, AVX2, CUDA v1/v2), and QAOA. We report how often each finds the exact best combination, how far its answers are from it, and, for QAOA, the measures of Section D.

E.3 Large problems

SA on all three platforms, plus exact search where possible. Every answer is re-scored by the judge.

E.4 What we measure

- **Answer quality**: best and median energy over runs; gap to the best answer $(E - E_{\text{opt}})/|E_{\text{opt}}|$; how often the best answer is found; spread across seeds.
- **Speed**: one-chain time (latency); steps per second with many chains (throughput); time until the best answer is reached (time to answer). For a solver that finds the best answer in a run with probability p_{succ}, the time to be 99% sure of finding it is $t_{\text{run}} \cdot \lceil \ln 0.01 / \ln(1 - p_{\text{succ}}) \rceil$.
- **Hardware use**: peak memory; achieved GPU memory bandwidth and occupancy (Nsight Compute).

- **Where the time goes**: data preparation, scoring, separation table, file building, solving and checking, each timed separately.

E.5 How we time things

Two warm-up runs, then the median of 10 runs, with the spread (interquartile range). CPU: the C++ monotonic clock (`steady_clock`). GPU: CUDA events, reported both kernel-only and end-to-end (including copying data to and from the GPU). Solver-only and total times are always reported separately. Nothing else runs on the machine during measurements, and the CPU frequency setting is recorded.

E.6 What stays the same across CPU, AVX2 and GPU, and what is tuned

Same everywhere: the problem, the algorithm, the temperature schedule, the number of sweeps, the random numbers, the arithmetic mode, and the judge. **Tuned per platform**: the number of chains run at once (each platform uses the number that minimizes its own time to answer), the GPU block size and kernel version, and loop unrolling. Latency and throughput comparisons use the same number of chains on every platform; time-to-answer comparisons use each platform's best setting. Both are reported, so the reader can see what was tuned.

E.7 Biological sanity checks (not proof that a combination works)

- Do the chosen combinations follow complementary exposure (every drug $z < 0$, every pair $s \geq 0$) more often than random k-drug teams from the shortlist? Reported with a permutation test.
- How do the chosen drugs change as λ goes from 0 ("the k drugs closest to AD") upward?
- Exploratory only: if the pipeline reviews [16, 1] list any drug pairs being tested together in AD trials, where do those pairs rank? There are too few such pairs for statistics.

E.8 Recommended figures and tables

F1 Pipeline diagram (modules, files, what runs where).

F2 Exact search: GPU speedup over CPU vs. n, one line per k.

F3 SA throughput (steps/s) vs. number of chains, scalar/AVX2/GPU v1/GPU v2, log–log.

F4 SA one-chain time and AVX2 speedup vs. n, with the share of accepted steps marked.

F5 SA time to answer vs. n on the three platforms.

F6 QAOA: optimum share and valid share vs. depth p, one line per n.

F7 QAOA: valid share vs. penalty size A.

F8 Where the time goes: stacked bar per pipeline stage.

F9 Chosen AD drugs vs. λ (heat map of how often each drug is chosen).

T1 Problem summary (n, k, share of nonzero P, $\binom{n}{k}$, best energy).

T2 Sequential vs. parallel timing table (required by the course).

T3 Test results T1–T12 (pass/fail).

T4 Top AD combinations with sources and caveats.

F Data, Provenance and Software Architecture

F.1 Data sources

Source	Used for	Access and terms
Cheng et al. 2019 supplementary data [5]	Interactome and drug–target table (default)	Open-access article supplement [TODO: confirm reuse terms of the supplementary files]
BioGRID [28], STRING [31]	Alternative interactomes (shared with sister project)	Free download; STRING under CC BY 4.0 [TODO: confirm BioGRID terms]
ChEMBL [29], DGIdb [30]	Supplementary drug–target evidence	ChEMBL under CC BY-SA 3.0; DGIdb free download [TODO: confirm DGIdb terms]
DrugBank [32]	Identifier cross-references only (optional)	Academic licence; raw data not redistributed in the repository
AD GWAS [15]	AD module	Published supplementary tables
AD pipeline reviews [16, 1]	Curated candidate list	Open-access articles

Table 10: Data sources. Raw files are never committed when their licence forbids it; the repository stores download scripts, version strings and SHA-256 checksums instead.

In plain words

Every number the solvers use must be traceable: either computed by a script from a cited source, set as a documented choice in a settings file, or generated randomly for testing (and labeled so). Nobody types in a biological number by hand.

Every numeric value in the objective is one of: *measured* (derived by a documented computation from a cited source), *assumption* (a modeling choice in the configuration, e.g. λ, $z_{\max}$), or *synthetic* (generated for testing, with the generator seed). There are no hand-assigned biological numbers.

F.2 Schemas

`candidates.csv` (one row per drug; the interface with the sister project)

```
drug_id          stable internal id (e.g., DB00843 or CHEMBL502)
name             preferred name
source_list      curated | sister | scaling_pool
targets          semicolon-separated Entrez Gene IDs present in the interactome
n_targets        integer
d_c, z, p_emp    proximity, z-score, empirical p-value (blank if missing)
n_random         random samples used for z
scorer           "sister@<git hash>" | "reference@<git hash>"
eligible         true/false;  ineligible_reason   free text code (NO_TARGETS, Z_NONNEG, MISSING_Z,
     NOT_IN_LCC)
provenance       reference to the rows of targets.csv and the scoring run id
```

`targets.csv` columns `drug_id`, `entrez_id`, `source_db`, `source_version`, `evidence_type`, `in_interactome`.

Instance (problem) files `instance.json` plus binary arrays:

```
{ "instance_id": "...", "family": "small|large|synthetic",
  "n": 20, "k": 3, "lambda": 1.0, "A": 0.53,   # penalty weight, used only by QAOA
  "drug_ids": [...],                        # row order = variable order = qubit order
  "files": {"w": "w.npy", "P": "P.npy", "w_int": "w_i32.npy", "P_int": "lamP_i32.npy"},
  "int_scale_bits": 20, "P_layout": "col-major, n_pad=24, 32-byte aligned",
  "sources": {"candidates": "<sha256>", "separation": "<sha256>", "config": "<sha256>"} }
```

`ising.json` h (dense), J (sparse list of (i, j, J_{ij})), offset C, scale factor, and the convention string `"x=(1-z)/2; measured 1 = selected; qubit i = row i of drug_ids"`.

Run records (`runs.jsonl`) One JSON object per run: instance id, solver, platform, mode, seed, chains, sweeps or QAOA settings, best set, solver energy, judge energy, valid, found-best, timings by stage (`t_prep`, `t_build`, `t_h2d`, `t_kernel`, `t_d2h`, `t_total`), counters (proposals, accepts, circuit evaluations, shots), peak memory, software versions, git hash.

Files saved along the way candidate table; shortlist; separation table; penalty table; QUBO and Ising files; temperature schedule and acceptance table; settings files; run records; final combination report (drug names, targets, z, pairwise s, sources, caveats).

F.3 Not changing the sister project's scores by accident

If the sister project supplies scores, its output file is only read, never written, and its SHA-256 hash and configuration are copied into every instance file. A regression test (T12) recomputes z for five drugs with the reference scorer at the same settings and fails if the values differ beyond Monte-Carlo tolerance, which detects silent changes in definitions on either side. No module in this project writes to the candidate scores.

F.4 Architecture and directory structure

The core is C++17/CUDA (performance-critical, required by the course) with NASM for the AVX2 kernel; orchestration, data preparation and QAOA are Python (CUDA-Q's primary interface and the sister project's tooling). Solvers are command-line programs that read an instance file and append run records, so every solver, including QAOA, has the same contract.

```
ad-combo/
  data/            raw/ (git-ignored, filled by scripts)  frozen/ (manifests, checksums)
  prep/            01_load_interactome.py  02_targets.py  03_ad_module.py
                   04_reference_scorer.py  05_shortlist.py  06_separation.py
  build/           07_build_instance.py  08_build_ising.py  09_quantize.py
  core/            include/{instance.hpp, record.hpp, philox.hpp, judge.hpp}
                   exact_cpu.cpp  sa_scalar.cpp  sa_avx2.cpp  asm/g_update_avx2.asm
                   intrin/g_update_intrin.cpp  cli/combo_solve.cpp
  cuda/            exact_search.cu  sa_v1.cu  sa_v2.cu
  quantum/         qaoa.py  decode.py  (stretch: qaoa_xy.py  dicke.py)
  bench/           configs/*.yaml  run_small.py  run_large.py  timing.py
  analysis/        figures.py  tables.py
```

```
tests/          test_formulation.py  test_ising.py  test_exact.cpp  test_sa_crossplatform.cpp
  ...
docs/           parallel_algorithms.md  results.md  (course deliverables)
```

F.5 Interfaces

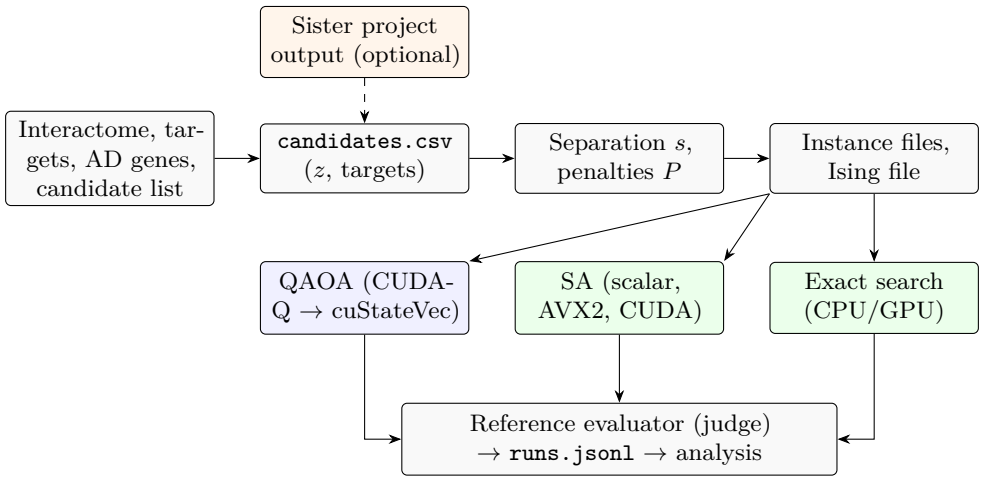
```
struct Instance { int n, k, n_pad; double lambda; double A;
                  std::vector<float> w, P;          // P column-major, padded
                  std::vector<int32_t> w_i, lamP_i; // deterministic mode
                  std::vector<std::string> drug_ids; std::string id, hash; };
struct SolverConfig { std::string solver;   // exact | sa
                      std::string platform; // scalar | avx2 | avx2_intrin | cuda_v1 | cuda_v2
                      Mode mode; uint64_t seed; int chains; int sweeps; double T0, T1; };
struct RunRecord { std::vector<int> best; double e_solver, e_judge; bool valid;
                   Timings t; Counters c; uint64_t seed; };
std::vector<RunRecord> solve(const Instance&, const SolverConfig&);
double judge(const Instance&, const std::vector<int>& selection);   // float64 reference
```

The Python side exposes `solve(instance_dir, config) -> list[RunRecord]`, calling the CLI for classical solvers and CUDA-Q for QAOA, and writes the same JSON record.

Configuration (YAML)

```
instance: instances/small_n20_k3_lam1.0
solver: sa
platform: cuda_v1
mode: deterministic
seed: 20261011
chains: 4096
sweeps: 2000
schedule: {type: geometric, T0: auto_80pct, T1: auto_0.1pct}
timing: {warmup: 2, repeats: 10}
```

F.6 Data flow



G Correctness and Regression Tests

These tests check that the code computes what the math says. They say nothing about whether a combination works biologically (Section E) or how fast the code is (Section E). CPU tests run on

every commit; GPU tests run on the GTX 1660 before every round of benchmarks. Each test has one owner.

ID	Test	What it does and when it passes	Owner
Scoring function			
T1	QUBO vs. direct energy	For all 2^n yes/no patterns ($n \leq 16$, Python), the QUBO energy equals the direct energy plus $A(\text{count} - k)^2$, to 10^{-9}.	M1
T2	Exactly k drugs	Every answer from exact search and SA has exactly k different drugs, all in range.	M3
T3	Safe penalty value	With A just above the proven value, the best of all 2^n patterns is valid and equals the exact-search answer, on every small AD problem and 1,000 random ones ($n \leq 16$). With A deliberately too small, a known counterexample is caught (shows the test can fail).	M1
T4	Integer rounding	Integer-mode best answer equals the double-precision best answer on every small problem.	M1
T5	QUBO $\rightarrow$ Ising	For all 2^n patterns ($n \leq 16$), the Ising energy (times scale, plus offset) equals the QUBO energy, to 10^{-6}.	M1
Solvers			
T6	Qubit order	Prepare known patterns in CUDA-Q, measure, decode; the decoded drugs must match.	M1
T7	Same decisions everywhere	Deterministic mode, fixed seed: scalar, AVX2 assembly, AVX2 intrinsics, CUDA v1 and CUDA v2 make identical accept/reject decisions for the first 10^6 steps of chains 0–63 and end in identical states.	M2, M3
T8	Exact search agrees	CPU and GPU exact search return the same best energy, tie count and first best combination; a brute-force Python check agrees for $n \leq 12$.	M2, M3
T9	Helper array stays correct	In debug builds, every 10^4 accepted swaps, rebuild g from scratch and compare with the updated g (must match exactly in integer mode).	M3
T10	Seeds	The same seed repeats a run exactly; different seeds give different runs.	M2
Data			
T11	Sister scores unchanged	Our script reproduces the sister project's z for five drugs within random-sampling tolerance; file checksums match the frozen list.	M1

ID	Test	What it does and when it passes	Owner
T12	Edge cases	Drug with no targets in the interactome → excluded; single-target drug → documented separation rule applied and flagged; all-zero P → answer is the k largest rewards; $k = 1$ and $k = n - 1$ work; ties counted correctly.	M1, M3

QAOA circuit checks (part of T5/T6): with all angles set to zero, the samples are spread evenly over all patterns (checked with a χ^2 test at $n = 10$).

H Limitations and Future Work

H.1 Limitations of the approach

- **Incomplete interactome.** Well-studied proteins have more known interactions, which pulls their drugs closer to everything. Comparing with random look-alike drugs (the z-score) reduces this bias but does not remove it.
- **The scoring rule is a proxy.** Complementary exposure was found in hypertension and cancer combinations [5]; we assume it carries over to AD. Our objective rewards that pattern; it does not predict efficacy, synergy or safety.
- **Weak validation.** Very few AD combinations have trial results, so we check patterns (complementary exposure, comparison with random teams), not outcomes.
- **Simulated quantum computer.** Results show answer quality on a noise-free simulator. They do not show how a real, noisy quantum computer would do, and they do not show any speed advantage.
- **One GPU and one CPU.** Speedups are specific to the GTX 1660 and our CPU; another machine would give different numbers, though the reasons behind them should carry over.

H.2 Features cut to keep the project finishable

The following were considered during planning and cut because three students, two of them new to CUDA and to optimization, have one term. Each is a natural next step, and the shared problem-file format means none of them would require changes to the solvers already built.

Cut feature	Why cut	How it would be added
Safety and effectiveness information	Needs several extra datasets, each with its own caveats; adds data work without adding parallel-computing content.	As extra terms in the same energy, so solvers stay unchanged: (1) known dangerous drug pairs from DDInter 2.0 [33] as forbidden swaps; (2) anticholinergic burden, which worsens cognition in older adults, from the ACB scale [34] as a per-drug penalty; (3) blood–brain-barrier permeability from B3DB [35] as a filter; (4) clinical-trial phase as a small bonus, switched off whenever trials are used for validation; (5) adverse-event signals from TWOSIDES [36] or SIDER [37] only as a sensitivity check, since they are noisy.
Splitting large problems for QAOA (decomposition)	QAOA is limited to about 28 drugs on this GPU; splitting is only needed beyond that, and it is a project of its own.	Keep the current best team, add $m - k$ other drugs, and solve that m-drug sub-problem exactly or with QAOA; repeat. Because the team is always inside the sub-problem, its score is exact and never gets worse. Related to qbsolv-style decomposition [38].
QAOA that never gives invalid answers (XY mixer, Dicke state)	The starting-state circuit is the hardest piece of quantum code in the plan.	Kept as a stretch goal (Section D); otherwise future work [21, 22, 23, 24].
GPU check of all 2^n yes/no patterns	A Python check up to $n = 16$ is enough to test the penalty proof; the GPU version would only extend it.	A Gray-code kernel (one bit changes per step) reusing the exact-search reduction.
Other search methods (parallel tempering, tabu search) and an SA version that uses the penalty	Each would need its own three implementations; one algorithm done carefully is a cleaner comparison.	Parallel tempering [18] reuses the SA kernels plus an exchange step between chains at different temperatures.
Other parallel layouts (8 chains per `ymm` register; multi-core OpenMP $\times$ AVX2)	Extra experiments, not needed for the course comparison.	The 8-chain layout needs `vpgatherdd` for the scattered reads and likely helps only for small n.

Cut feature	Why cut	How it would be added
Robustness to other AD gene lists or other interactomes	Data work rather than parallel-computing work.	Re-run the pipeline with the sister project's alternative modules and graphs; compare chosen combinations.
Real quantum hardware; "at most k" drugs; synergy and dose prediction	No access; or no reliable AD data.	Out of scope for this project.

I Risks and Decision Matrix

I.1 Risks and what we do about them

Risk	Impact	What we do
CUDA is new to two members	Medium	Week 7 ramp-up exercises; start with the simplest kernel (v1, one thread per chain); test T7 compares every step with the scalar version, so bugs show up immediately; Members 2 and 3 pair on the SA kernel.
Only Member 1 knows the quantum part	Medium	Quantum work is not graded core; Ising files and tests are finished by Week 9, so QAOA can be reported as future work if Member 1 runs out of time.
CUDA-Q does not install or cannot see the GPU (driver, WSL2)	High for RQ3 only	Check in Week 6; fallback to a native Linux boot, CUDA-Q's CPU simulator for small n, or cuQuantum Python. RQ1–RQ2 unaffected.
Sister project output late or never	Low	Decision D1: curated list plus our own scoring script.
Assembly bugs	Low	The team knows x86; intrinsics twin and tests T7, T9 cross-check.
SIMD/GPU gains smaller than hoped	Medium (a result, not a failure)	Explain with the share of accepted steps and Nsight numbers; report throughput with many chains.
QAOA slow to tune at $n \geq 20$	Medium	Prioritize $n \leq 16$; reuse tuned angles across problems; cap evaluations.
Over-reading the biology	High (credibility)	Caveats in every results table; no claims of efficacy or safety.

Table 13: Risks.

I.2 Minimum version vs. full version

Minimum version (passes the course): checked scoring function; exact search on CPU and GPU; SA in scalar C++, AVX2 assembly and CUDA v1 with identical decisions; timing tables on small and large problems; the best AD combinations. **Full version (supports a short paper):** plus CUDA v2 and standard QAOA measured against exact answers. **Stretch:** XY-mixer QAOA.

I.3 Decision matrix

Scores are 1 (low) to 5 (high). "Course" is the value for the SIMD/SIMT grading criteria; "Novelty" is research value. The build order puts doability first, then novelty.

Component	Doable	Course	Novelty	Order	Reason
Problem builder + tests	5	3	2	1	Everything depends on it.
Exact search (CPU, GPU)	5	4	2	2	Answer key; a good first CUDA kernel.
Scalar SA + deterministic mode	5	3	2	3	Reference for all other versions.
AVX2 assembly SA	5	5	2	4	Core SIMD deliverable; team already knows `ymm`.
CUDA SA v1 (thread per chain)	4	5	3	5	Core SIMT deliverable.
CUDA SA v2 (warp per chain)	3	4	3	6	Better memory use; interesting comparison with v1.
Standard QAOA (CUDA-Q)	3	3	4	7	Main research result.
XY-mixer QAOA (stretch)	2	2	5	8	Strongest novelty, hardest circuit.

Table 14: Decision matrix: build in the order of the "Order" column.